A CASE-STUDY BASED REPORT

ON

Machine Learning Based Crop Yield and Fertiliser Recommendation System for Sustainable Farming

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

P. Sushmitha **Roll No. 2520030073**

KBindusree **Roll No. 2520030434**

Under the Esteemed guidance of

Dr. Raghavendra

Faculty Guide, Department of CSE



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Introduction.....

Background.....
Motivation for the Project.....
Overview of the Work Carried Out.....
Scope and Responsible-Use Statement.....

Problem Statement.....

Case-Study Scenario.....
Central Case-Study Question.....
Formal Problem Definition.....
Objectives that Guide the Study.....
Assumptions and Limitations.....

Dataset and Exploratory Data Analysis

About the Dataset.....
Purpose of EDA.....
Preparation Steps Actually Performed.....
EDA Exploration - Implementation.....
Summary of EDA Findings.....

Algorithm Selection and Implementation.....

Conclusion

Signature of the Team members

ABSTRACT

Agriculture depends on selecting suitable crops and maintaining balanced soil nutrients. This case study presents a machine-learning application that recommends a crop and a broad fertiliser category from soil nutrient values and environmental conditions. The system is implemented using Python, Streamlit, Pandas and Scikit-learn. A Random Forest Classifier is trained on crop_recommendation.csv using nitrogen (N), phosphorus (P), potassium (K), soil pH, rainfall and temperature as input features. The application provides two interfaces: Simple Farmer Mode, which maps general farm answers to representative values, and Soil Test Mode, which accepts numeric soil-test values. After prediction, the application compares the entered N, P and K values with the median values for the predicted crop in the dataset and suggests a broad fertiliser category. The model is evaluated using a stratified 80:20 train-test split and accuracy score. The system is intended as a decision-support prototype; actual field recommendations should be checked against local agronomic advice and soil-test reports.

1. INTRODUCTION

1.1 Background

Crop selection is affected by soil nutrient availability, soil acidity or alkalinity, rainfall and temperature. Choosing a crop without considering these conditions may reduce productivity or waste resources. Machine learning can identify patterns in historical crop data and use those patterns to suggest a crop for a new set of conditions.

1.2 Motivation for the Project

The project aims to make crop recommendation easier to access. Some farmers may not be familiar with technical soil-test terms, while others may already have laboratory results. The application therefore provides a simple question-based mode and a numeric soil-test mode in one interface.

1.3 Overview of the Work Carried Out

- Loaded the crop_recommendation.csv dataset with Pandas.
- Selected N, P, K, pH, rainfall and temperature as model features and crop as the target label.
- Split the data into training and testing sets using an 80:20 ratio, random_state=42 and stratification by crop.
- Trained a RandomForestClassifier with 100 trees and evaluated it using accuracy_score.
- Built a Streamlit interface with Simple Farmer Mode and Soil Test Mode.
- Generated a broad fertiliser-category suggestion by comparing the user's N, P and K values with the predicted crop's dataset medians.

1.4 Scope and Responsible-Use Statement

The current prototype recommends a crop label and a general fertiliser category. It does not directly predict crop yield in kilograms or tonnes, and it does not calculate an exact fertiliser dose. Simple Farmer Mode uses representative values coded in the application; soil type, season and water availability are not independent training features. Model confidence is the largest predicted class probability and should not be interpreted as a guarantee of real-world success. Users should verify recommendations with soil testing, local climate information and qualified agricultural experts.

2. PROBLEM STATEMENT

2.1 Case-Study Scenario

A farmer wants to identify a suitable crop using the farm's nutrient levels and environmental conditions. The farmer may either enter numeric measurements from a soil test or answer basic questions about soil, rainfall and temperature. The application must process the supplied information and display a crop recommendation and a broad fertiliser category.

2.2 Central Case-Study Question

How can a machine-learning classifier use N, P, K, soil pH, rainfall and temperature to recommend a crop, and how can the same application provide a simple nutrient-based fertiliser category for the user?

2.3 Formal Problem Definition

Let each observation be represented by X = [N, P, K, pH, rainfall, temperature]. The target variable y is the crop label. The supervised learning task is to learn a function f(X) that predicts the crop class. After predicting a crop, the application obtains the median N, P and K values for that crop from the dataset, calculates the difference between each median and the user's entered nutrient value, and identifies the largest difference. If all differences are non-positive, it returns "Balanced NPK fertilizer"; otherwise, it returns a broad nitrogen-rich, phosphorus-rich or potassium-rich category.

2.4 Objectives that Guide the Study

- Develop a crop classifier using the supplied dataset.
- Accept farm details through a user-friendly Streamlit interface.
- Support both general farmer inputs and numeric soil-test inputs.
- Display the predicted crop and model confidence.
- Provide a basic fertiliser-category suggestion from N, P and K comparisons.
- Show the model test accuracy to support basic evaluation.

2.5 Assumptions and Limitations

- The CSV data are treated as representative of the crop classes in the dataset.
- The model uses six features only; it does not use soil type, season, water availability or location as independent model inputs.
- Simple Farmer Mode maps choices to fixed representative values for demonstration.
- The test accuracy depends on the supplied dataset and the chosen random split.
- The fertiliser logic gives a broad category, not a product, dosage, application schedule or field-specific prescription.
- The app's title includes "yield", but the current code predicts crop class rather than numerical crop yield.

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 About the Dataset

The supplied crop_recommendation.csv file contains 1400 observations and 7 columns: N, P, K, pH, rainfall, temperature and crop. The six numeric columns are input features, while crop is the categorical target. The dataset contains 14 crop classes, with 100 to 100 records per class. In this file, each of the 14 crop labels has 100 records.

3.2 Purpose of EDA

Exploratory Data Analysis (EDA) helps check data quality, understand feature ranges, examine class balance and identify unusual or missing values before training. It also helps describe what information the model can learn from the dataset.

3.3 Preparation Steps Actually Performed

The provided app.py loads the CSV with pd.read_csv, selects the six feature columns and assigns the crop column as the target. It then uses train_test_split with test_size=0.20, random_state=42 and stratify=y. The supplied code does not explicitly perform imputation, scaling, duplicate removal or feature engineering; the dataset was checked separately for this report to summarise its basic quality.

3.4 EDA Exploration – Implementation

- Dataset size: 1400 rows and 7 columns.
- Missing values: 0 across all columns.
- Duplicate rows: 0.
- Crop classes: 14, with 100 observations for each class.
- N values range from 10.1 to 139.8; P from 5.0 to 99.9; K from 5.2 to 59.4.
- Soil pH ranges from 5.00 to 7.99; rainfall from 40.1 to 299.5; temperature from 15.0 to 35.0.

3.5 Summary of EDA Findings

The dataset is complete for the seven listed columns and contains no exact duplicate rows. Crop classes are evenly represented, which supports use of stratified train-test splitting. Numeric feature ranges differ substantially, but the current Random Forest model does not require standard scaling. The dataset statistics describe the supplied records only and do not establish that every recommendation is suitable for every local farm.

4. ALGORITHM SELECTION AND IMPLEMENTATION

4.1 Random Forest Classifier

The implementation uses RandomForestClassifier from Scikit-learn with n_estimators=100 and random_state=42. A Random Forest combines predictions from multiple decision trees. For classification, the trees vote for crop classes, and the classifier returns the selected class. It can model non-linear relationships between nutrient values and environmental conditions without requiring feature scaling.

4.2 Training and Evaluation

The code divides the dataset into 80% training data and 20% test data. The stratify=y parameter maintains crop-class proportions in both sets. The model is fitted on the training portion, then predicts the test portion. accuracy_score calculates the fraction of test labels predicted correctly. The actual accuracy is calculated when the app runs and is displayed in the “About the ML Model” expander; no fixed accuracy value is assumed in this report.

4.3 Crop Recommendation Workflow

- Load the CSV and define the six model features.
- Train the Random Forest model and calculate test accuracy.
- Ask the user to choose Simple Farmer Mode or Soil Test Mode.

- Convert simple choices to representative N, P, K, pH, rainfall and temperature values, or accept numeric values directly.
- Predict the crop and obtain the largest class probability as the displayed confidence.
- Compare user N, P and K values with the median nutrient values for the predicted crop.
- Display the crop, fertiliser category and relevant farm information.

4.4 Fertiliser Recommendation Logic

For the predicted crop, the code calculates the median N, P and K values from its dataset rows. It subtracts the user's respective nutrient values from these medians. The nutrient with the largest difference determines the category, provided that the largest difference is positive. If no nutrient difference is positive, the app displays a balanced NPK category. This is a simplified heuristic and does not replace a fertiliser prescription based on soil-test interpretation and local recommendations.

4.5 Tools and Requirements

- Python – programming language.
- Streamlit – interactive web application interface.
- Pandas – loading and handling the CSV dataset.
- Scikit-learn – train-test split, Random Forest classification and accuracy evaluation.
- Dataset file – crop_recommendation.csv, stored in the same project directory as app.py.

The stated requirements.txt dependencies are streamlit, pandas and scikit-learn. These packages should be installed in the Python environment used to run the application.

5. CONCLUSION

The Crop Yield & Fertiliser Recommendation project demonstrates how a supervised machine-learning model can support crop selection from nutrient and environmental measurements. The application combines a Random Forest classifier with a Streamlit interface and offers two input modes for different users. The supplied dataset contains 1,400 records across 14 balanced crop classes, with no missing values or duplicate rows found in the file reviewed for this report. The system also presents a simple fertiliser-category heuristic and test accuracy. Future improvements could include external validation with local agricultural data, explicit yield prediction using a suitable yield-labelled dataset, feature-importance analysis, more rigorous model evaluation and expert-verified fertiliser dosage recommendations.

REFERENCES

- Project repository: https://github.com/sushmitha290807/Machine-learning---Crop-Yield-Fertiliser-Recommendation
- Scikit-learn documentation: RandomForestClassifier and model_selection.train_test_split.
- Streamlit documentation: app interface components and caching.
- Project files supplied for this report: app.py, crop_recommendation.csv and requirements.txt dependency list.